```
==  CreateArticles: migrating
================================================
-- create_table(:articles)
   -> 0.0019s
==  CreateArticles: migrated (0.0020s)
=======================================
```

Because you're working in the development environment by default, this command will apply to the database defined in the `development` section of your `config/database.yml`file. If you would like to execute migrations in another environment, for instance in production, you must explicitly pass it when invoking the command: `rake db:migrate RAILS_ENV=production`.

5.6 Saving data in the controller

Back in `ArticlesController`, we need to change the `create` action to use the new `Article` model to save the data in the database. Open `app/controllers/articles_controller.rb` and change the `create` action to look like this:

```
def create
  @article = Article.new(params[:article])

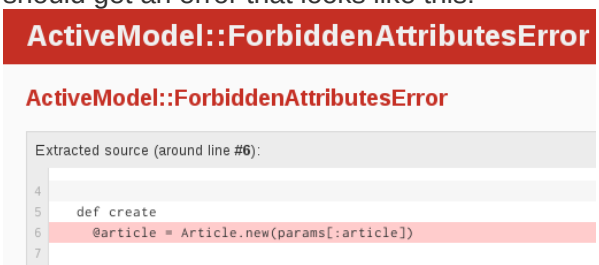
  @article.save
  redirect_to @article
end
```

Here's what's going on: every Rails model can be initialized with its respective attributes, which are automatically mapped to the respective database columns. In the first line we do just that (remember that `params[:article]` contains the attributes we're interested in). Then, `@article.save` is responsible for saving the model in the database. Finally, we redirect the user to the `show` action, which we'll define later.

You might be wondering why the `A` in `Article.new` is capitalized above, whereas most other references to articles in this guide have used lowercase. In this context, we are referring to the class named `Article` that is defined in `app/models/article.rb`. Class names in Ruby must begin with a capital letter.

As we'll see later, `@article.save` returns a boolean indicating whether the article was saved or not.

If you now go to http://localhost:3000/articles/new you'll *almost* be able to create an article. Try it! You should get an error that looks like this:



Rails has several security features that help you write secure applications, and you're running into one of them now. This one is called strong parameters, which requires us to tell Rails exactly which parameters are allowed into our controller actions.

Why do you have to bother? The ability to grab and automatically assign all controller parameters to your model in one shot makes the programmer's job easier, but this convenience also allows malicious use. What if a request to the server was crafted to look like a new article form submit but also included extra fields with values that violated your applications integrity? They would be 'mass assigned' into your model and then into the database along with the good stuff - potentially breaking your application or worse.